

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report, cohen_kappa_score
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.metrics import roc_curve, roc_auc_score, RocCurveDisplay
```

6. Separação dos dados

6.1 Utilizando todas as features

```
x = df.drop(columns = ['quality_binary', 'quality'])
y = df['quality_binary'] # o que eu quero prever. (target)
```

```
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, stratify=y, random_state=42)
```

6.2 Utilizando features mais correlacionadas

```
x_corr = df[['alcohol', 'volatile acidity', 'sulphates', 'citric acid']]
y_corr = df['quality_binary'] # o que eu quero prever. (target)
```

```
x_train_corr, x_test_corr, y_train_corr, y_test_corr = train_test_split(x_corr, y, test_size=0.2, stratify=y, random_state=42)
```

7. Normalizando os dados

Normalizando todas as variáveis

```
#Normalizar apenas com os dados de treino para não ocorrer vazamento de dados e colocado "FIT" para os dados de test pelo n
scaler = StandardScaler()
scaler.fit(x_train)
x_train_escalonado = scaler.transform(x_train) # X_TRAIN → Aprende + Transforma. O scaler "estuda" a média e desvio padrão
x_test_escalonado = scaler.transform(x_test) # X_TEST → Só Transforma. O scaler usa os parâmetros que APRENDEU do X_train
```

Normalizando variáveis correlacionadas

```
#Normalizar apenas com os dados de treino para não ocorrer vazamento de dados e colocado "FIT" para os dados de test pelo n
scaler = StandardScaler()
scaler.fit(x_train_corr)
x_train_escalonado_corr = scaler.transform(x_train_corr) # X_TRAIN → Aprende + Transforma. O scaler "estuda" a média e desv
x_test_escalonado_corr = scaler.transform(x_test_corr) # X_TEST → Só Transforma. O scaler usa os parâmetros que APRENDEU de
```

8. MODELO DE MACHINE LEARNING

8.1 KNN

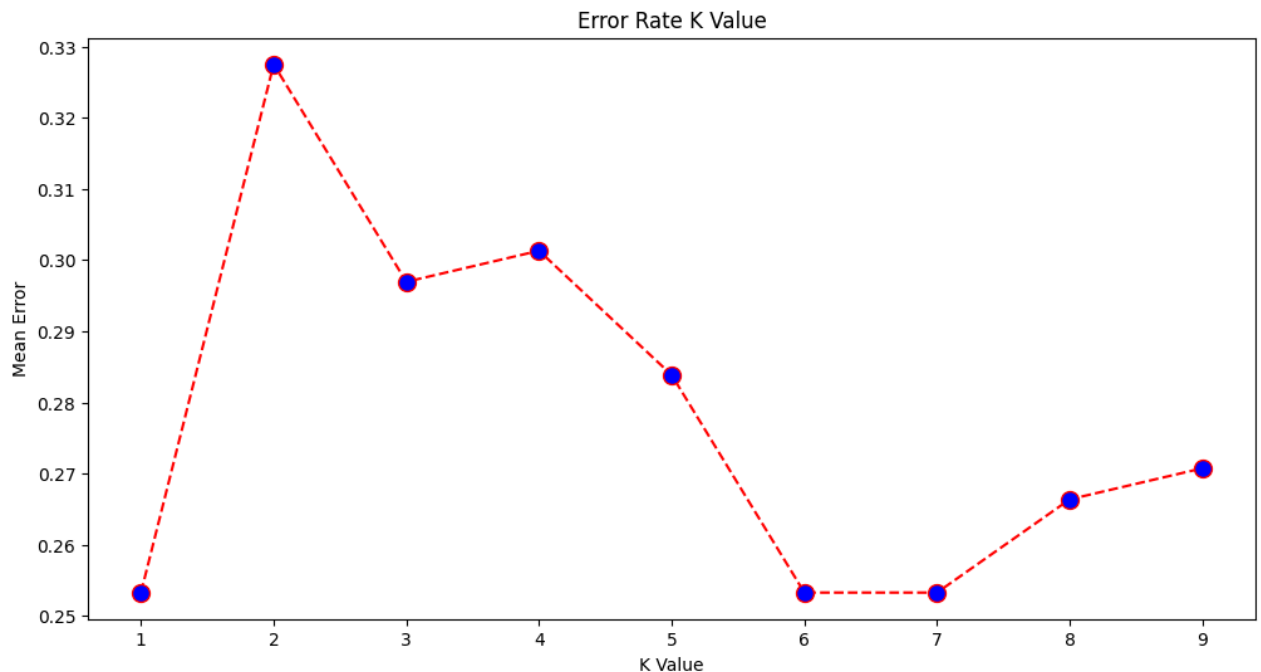
8.1.1 Criando Modelo - 11 Variáveis

```
#Melhorando Hiperparâmetros
error = []

# Calculating error for K values between 1 and 10
for i in range(1, 10): #range de tentativas para k
    knn = KNeighborsClassifier(n_neighbors=i) # aqui definimos o k
    knn.fit(x_train_escalonado, y_train) #treinando o algoritmo para encontrar o erro
    pred_i = knn.predict(x_test_escalonado) #armazenando as previsões
    error.append(np.mean(pred_i != y_test)) #armazenando o valor do erro médio na lista de erros
```

```
plt.figure(figsize=(12, 6))
plt.plot(range(1, 10), error, color='red', linestyle='dashed', marker='o',
         markerfacecolor='blue', markersize=10)
plt.title('Error Rate K Value')
plt.xlabel('K Value')
plt.ylabel('Mean Error')
```

Text(0, 0.5, 'Mean Error')



Vamos usar uma técnica chamada Gridsearch que é basicamente a busca por força bruta
 # Iremos utilizar a técnica de validação cruzada de 5 folds (divisões) em cima do conjunto de treinamento
 # Como métrica de melhor desempenho usaremos a acurácia, isto é, estamos buscando os hiperparâmetros que maximizam a acurácia

```
from sklearn.model_selection import GridSearchCV #metodo para seleção dos melhores Ks (#Basicamente a busca por força bruta)
from sklearn.metrics import make_scorer, accuracy_score, f1_score #métricas de validação
import numpy as np
import matplotlib.pyplot as plt
```

```
# Parâmetros testados
param_grid = {'n_neighbors':[6,7,8,9], #total de vizinhos
              'weights': ['uniform', 'distance'], #Função de peso usada na previsão
              'metric':['cosine', 'euclidean', 'manhattan'] #métrica para cálculo da distância
            }
```

```
# Métrica de desempenho = melhor acurácia
gs_metric = make_scorer(accuracy_score, greater_is_better=True)
```

```
grid = GridSearchCV(KNeighborsClassifier(),
                    param_grid=param_grid,
                    scoring=gs_metric,
                    cv=5, n_jobs=4, verbose=3) #cv = gerador de validação cruzada / n_jobs = determina a quantidade de jobs
```

```
grid.fit(x_train_escalonado, y_train) #Selecionando os melhores hiperparametros com base nos dados de treino
knn_params = grid.best_params_
print('KNN', knn_params)
```

Fitting 5 folds for each of 24 candidates, totalling 120 fits
 KNN {'metric': 'cosine', 'n_neighbors': 9, 'weights': 'distance'}

```
knn = KNeighborsClassifier(n_neighbors=9, metric='cosine', weights='distance') # utilizado o parametro de grid como melhor
knn.fit(x_train_escalonado, y_train)
```

```
KNeighborsClassifier
KNeighborsClassifier(metric='cosine', n_neighbors=9, weights='distance')
```

```
#prever
y_pred = knn.predict(x_test_escalonado)
```

```
#Avaliar
print(f"Accuracy: {accuracy_score(y_test, y_pred)}")
```

Accuracy: 0.7860262008733624

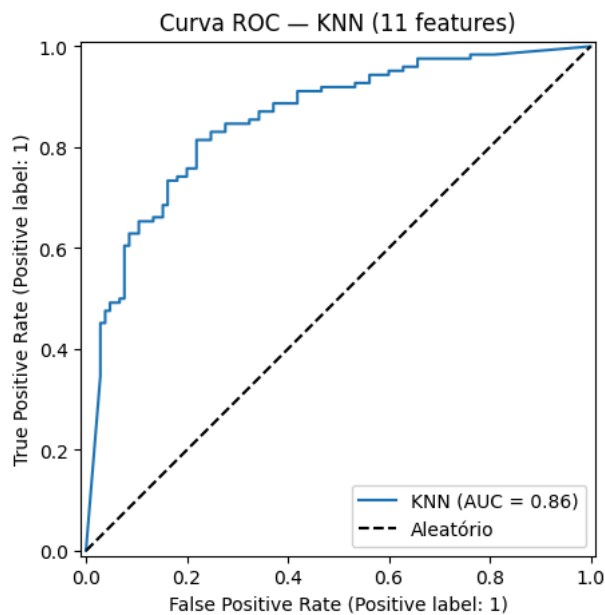
8.1.2 CURVA ROC - 11 Variáveis

```
# Criando curva ROC
```

```
y_score_knn = knn.predict_proba(x_test_escalonado)[: , 1]
print(f"AUC: {roc_auc_score(y_test, y_score_knn):.4f}")
```

```
fig, ax = plt.subplots(figsize=(6, 5))
RocCurveDisplay.from_predictions(y_test, y_score_knn, name='KNN', ax=ax)
ax.plot([0, 1], [0, 1], 'k--', label='Aleatório')
ax.set_title('Curva ROC - KNN (11 features)')
ax.legend()
plt.show()
```

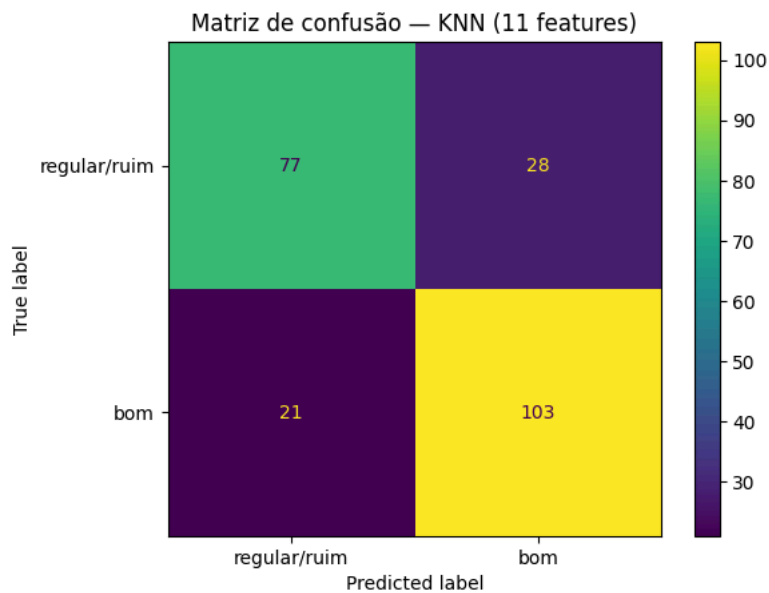
AUC: 0.8565



8.1.3 Matriz de Confusão - 11 Variáveis

```
from sklearn.metrics import ConfusionMatrixDisplay
matriz_confusao = confusion_matrix(y_true=y_test,
                                    y_pred=y_pred,
                                    labels=[0, 1])

disp = ConfusionMatrixDisplay(confusion_matrix=matriz_confusao,
                              display_labels=['regular/ruim', 'bom'])
disp.plot(values_format='d')
plt.title('Matriz de confusão - KNN (11 features)')
plt.show()
```



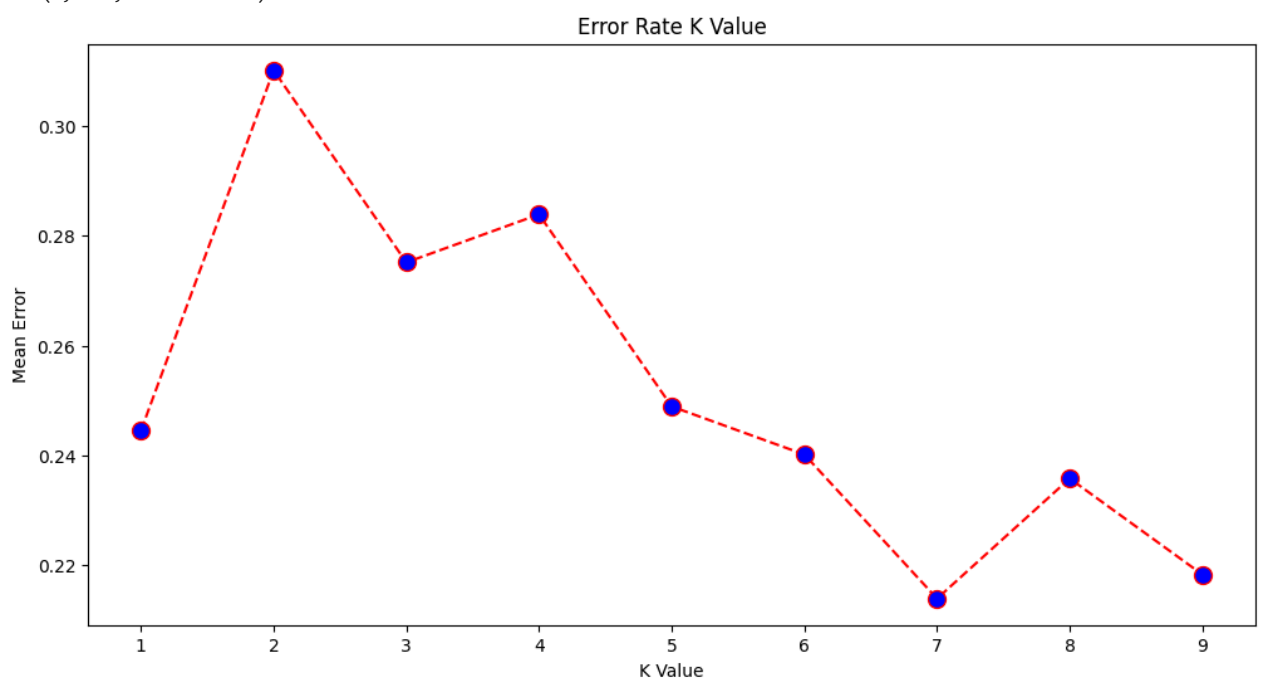
8.1.4 Criando modelo com 4 váriavies mais correlacionadas

```
#Melhorando Hiperparâmetros
error = []

# Calculating error for K values between 1 and 10
for i in range(1, 10): #range de tentativas para k
    knn = KNeighborsClassifier(n_neighbors=i) # aqui definimos o k
    knn.fit(x_train_escalonado_corr, y_train_corr) #treinando o algoritmo para encontrar o erro
    pred_i = knn.predict(x_test_escalonado_corr) #armazenando as previsões
    error.append(np.mean(pred_i != y_test)) #armazenando o valor do erro médio na lista de erros

plt.figure(figsize=(12, 6))
plt.plot(range(1, 10), error, color='red', linestyle='dashed', marker='o',
         markerfacecolor='blue', markersize=10)
plt.title('Error Rate K Value')
plt.xlabel('K Value')
plt.ylabel('Mean Error')
```

Text(0, 0.5, 'Mean Error')



```
param_grid_corr = {'n_neighbors':[7,8,9], #total de vizinhos
```

```
'weights': ['uniform', 'distance'], #Função de peso usada na previsão
'metric':['cosine', 'euclidean', 'manhattan'] #métrica para cálculo da distância
}
```

```
# Métrica de desempenho = melhor acurácia
gs_metric_corr = make_scorer(accuracy_score, greater_is_better=True)

grid_corr = GridSearchCV(KNeighborsClassifier(),
                          param_grid=param_grid_corr,
                          scoring=gs_metric_corr,
                          cv=5, n_jobs=4, verbose=3) #cv = gerador de validação cruzada / n_jobs = determina a quantidade de jobs

grid_corr.fit(x_train_escalonado_corr, y_train_corr) #Selecionando os melhores hiperparametros com base nos dados de treino
knn_params_corr = grid_corr.best_params_
print('KNN', knn_params_corr)
```

```
Fitting 5 folds for each of 18 candidates, totalling 90 fits
KNN {'metric': 'manhattan', 'n_neighbors': 9, 'weights': 'distance'}
```

```
knn_corr = KNeighborsClassifier(n_neighbors=9, metric='manhattan', weights='distance') # utilizado o parametro de grid com
knn_corr.fit(x_train_escalonado_corr, y_train_corr)
```

```
▼ KNeighborsClassifier ⓘ ?
KNeighborsClassifier(metric='manhattan', n_neighbors=9, weights='distance')
```

```
y_pred_corr = knn_corr.predict(x_test_escalonado_corr)
```

```
print(f"Accuracy: {accuracy_score(y_test_corr, y_pred_corr)}")
```

```
Accuracy: 0.8034934497816594
```

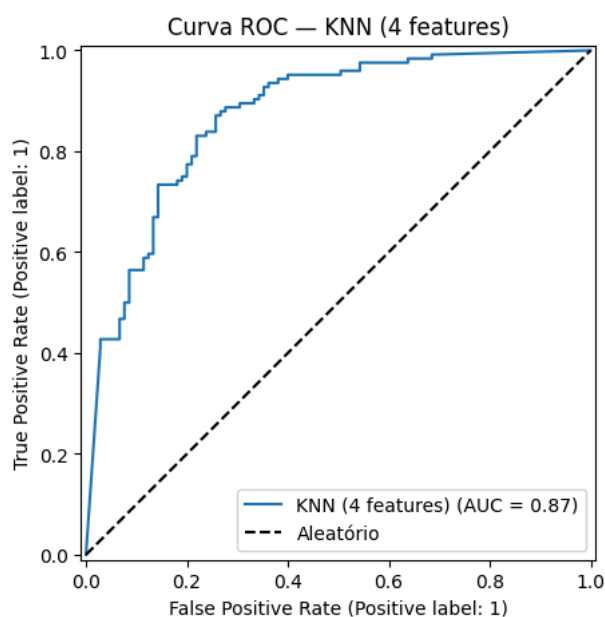
8.1.5 Curva ROC 4 váriavies mais correlacionadas

```
# Criando curva ROC - 4 features

y_score_knn_corr = knn_corr.predict_proba(x_test_escalonado_corr)[: , 1]
print(f"AUC: {roc_auc_score(y_test_corr, y_score_knn_corr):.4f}")

fig, ax = plt.subplots(figsize=(6, 5))
RocCurveDisplay.from_predictions(y_test_corr, y_score_knn_corr,
                                name='KNN (4 features)', ax=ax)
ax.plot([0, 1], [0, 1], 'k--', label='Aleatório')
ax.set_title('Curva ROC - KNN (4 features)')
ax.legend()
plt.show()
```

```
AUC: 0.8725
```



8.1.6 Matriz de Confusão 4 váriavies mais correlacionadas

```
matriz_confusao = confusion_matrix(y_true=y_test_corr,
```

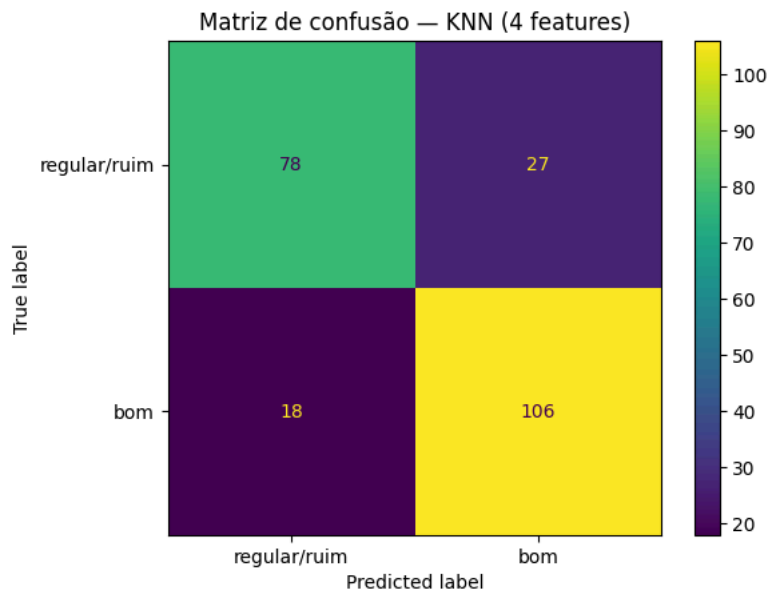
```

y_pred=y_pred_corr,
labels=[0, 1])

figure = plt.figure(figsize=(6, 5))
disp = ConfusionMatrixDisplay(confusion_matrix=matriz_confusao,
                             display_labels=['regular/ruim', 'bom'])
disp.plot(values_format='d')
plt.title('Matriz de confusão - KNN (4 features)')
plt.show()

```

<Figure size 600x500 with 0 Axes>



8.1.7 Observações - KNN

-- Accuracy

O KNN foi testado com dois conjuntos de variáveis. Com as 11 variáveis físico-químicas, a acurácia foi de 78,6%; com as 4 de maior correlação, 80,3%. Ambos bem acima da baseline de 54,2%.

-- **Matriz de Confusão** As matrizes de confusão mostram que o modelo erra mais ao aprovar do que ao reprovar: com 4 variáveis, foram 27 vinhos regulares classificados como bons contra 18 bons classificados como regulares. Ou seja, é mais permissivo que restritivo.

A diferença de 1,7 ponto entre os conjuntos equivale a 4 vinhos de 229 e não é significativa. A vantagem do conjunto reduzido é prática: mesmo desempenho medindo 4 variáveis em vez de 11.

Como limitação, o dataset mantém 125 linhas duplicadas (10,9%, seção 2.4), o que tende a tornar as métricas do KNN levemente otimistas.

-- Curva ROC

O AUC foi de 0,857 com 11 variáveis e 0,873 com 4. Ambos bem acima de 0,5, confirmando boa separação das classes.

O AUC mede se o modelo da nota maior a um vinho bom do que a um ruim, independentemente do corte. A acurácia depende do corte fixo em 0,5. Por medirem coisas diferentes, não devem ser comparadas entre si. Os dois conjuntos separam igualmente bem, o com 4 variáveis mais correlacionadas é preferível por praticidade, não por precisão.

Comece a programar ou gere código com IA.